

Organização e Arquitetura de Computadores

Jorge Augusto Salgado Salhani

no. USP: 8927418

Maio, 2026

1 Lista 3

Para esta lista, vamos considerar o caminho de dados mostrado abaixo (Sarah and David Harris's Digital Design and Computer Architecture, RISC-V Edition)

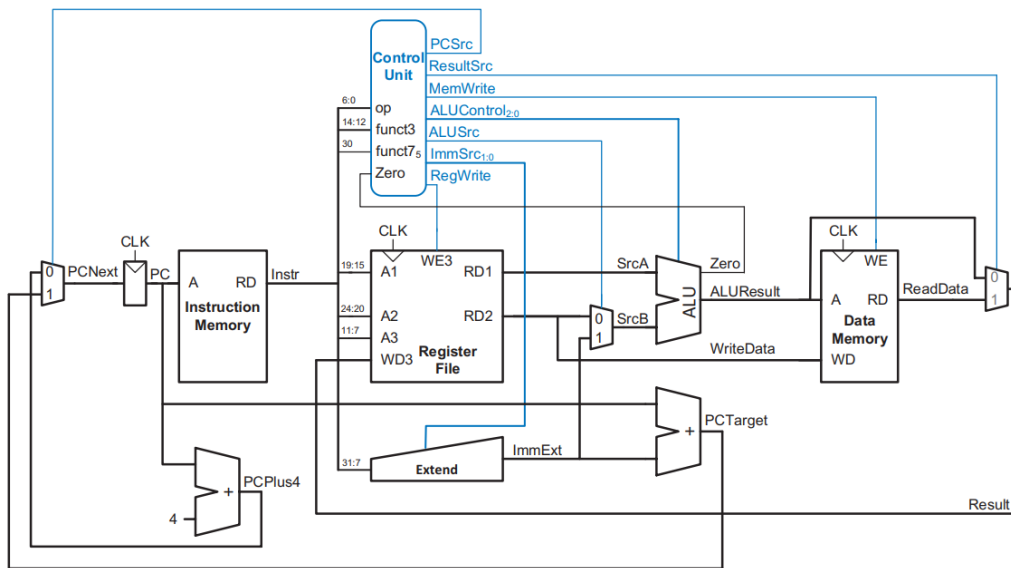


Figure 7.12 Complete single-cycle processor

Table 7.2 Main Decoder truth table

Instruction	Op	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp
lw	0000011	1	00	1	0	1	0	00
sw	0100011	0	01	1	1	x	0	00
R-type	0110011	1	xx	0	0	0	0	10
beq	1100011	0	10	0	0	x	1	01

1. Considere a seguinte ideia: modificar a arquitetura do conjunto de instruções e retirar a capacidade de

especificar um deslocamento de instruções de acesso à memória. Especificamente, todas as instruções *load-store* com deslocamento diferente de zero devem tornar-se pseudo-instruções e devem ser implementadas por meio de duas instruções. Por exemplo:

```
addi $at, $t1, 104    # some o deslocamento de um registrador temporário
lw $t0, $at           # nova forma de fazer lw $t0, 104($t1)
```

Que mudanças devem ser feitas no caminho de dados monociclo e no controle para que essa arquitetura simplificada possa funcionar?

[SOLUÇÃO]

Considerando o caminho de dados monociclo, se não iremos especificar valores imediatos como deslocamento de instrução (imediatos serão utilizados, como **addi**, porém não como offset) não haverá cálculo do novo endereço de memória pela ALU, portanto podemos remover a ALU geradora de **ALUResult** (lembrando que originalmente **SrcA** indica o endereço base e **SrcB**, o valor imediato utilizado para calcular o deslocamento).

Pela mudança na ALU, o sinal de controle **ALUSrc** torna-se dispensável.

2. Considere a arquitetura RISC-V Monociclo final implementada em sala de aula. Implemente as alterações necessárias para que essa arquitetura execute, além das instruções já implementadas, a instrução **addi** **<rd>**, **<rsi>**, **imm**. Para a sua implementação altere o que for necessário diretamente no caminho de dados monociclo e na tabela da UC vista em sala de aula. Caso você conclua que não precisam ser alterados, mostre porque, definindo as conexões usadas para a execução da instrução.

[SOLUÇÃO]

Para adicionar uma nova operação, vamos tomar como referência a tabela verdade.

Para a instrução **addi** temos

- Escrita em registradores destino, portanto **RegWrite** = 1
- O valor imediato é escolhido como os bits [31:20], portanto **ImmSrc** = 00
- O registrador destino é informado na instrução e vem de SrcB, portanto **ALUSrc** = 1
- Não há escrita em memória, logo **MemWrite** = 0
- O resultado que será escrito no registrador destino vêm de cálculo, e não da memória, logo **ResultSrc** = 0
- A operação não é desvio, portanto **Branch** = 0
- A operação da ALU é de soma, portanto **ALUOp** = 00

Dessa forma, a tabela da UC para a instrução **addi** é dada por

Instruction	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp
addi	1	00	1	0	0	0	00

3. Considere arquitetura RISC-V Monociclo final implementada em sala de aula. Implemente as alterações necessárias para que essa arquitetura execute, além das instruções já implementadas, a instrução **jal**, **<rd>**, **offset**. Para a sua implementação altere o que for necessário diretamente no caminho de dados monociclo e na tabela da UC vista em sala de aula. Caso você conclua que não precisam ser alterados, mostre porque definindo as conexões usadas para a execução da instrução.

[SOLUÇÃO]

Para a instrução **jal** precisamos calcular o endereço de destino (com deslocamento em **PC**) e salvar a instrução que seria a próxima, caso não houvesse o salto (**PC + 4**).

Para a instrução **jal** o campo **PCNext** deve ser calculado com base no valor imediato, similar à **j**. Portanto, sem mudanças no caminho de dados original.

No entanto, para que seja possível salvar **PC + 4** em registrador, é necessário que o resultado da ALU para **PC + 4** seja multiplexado via **ResultSrc**, que, neste caso, conduzirá esse resultado a **WD3**.

Para a tabela verdade de UC, temos

- Escrita em registrador, logo **RegWrite = 1**
- Nova opção para escrita no registrador destino, **WD3**, logo **ResultSrc = 10**
- Escolhemos os bits [12], [19:12], [20], [30:21], 1 (assinatura **J**), logo **ImmSrc = 11**
- Não utilizamos ALU para cálculo de endereço, logo **ALUSrc = ALUOp = X**
- Não escrevemos em memória, logo **MemWrite = 0**
- Não utilizamos desvio, logo **Branch = 0**
- Precisamos atribuir novo valor a **PCNext** como o imediato, logo **Jump = 1**

Instruction	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp	Jump
jal	1	11	X	0	10	0	XX	1

4. Considere a arquitetura RISC-V Monociclo final implementada em sala de aula. Implemente as alterações necessárias para que essa arquitetura execute, além das instruções já implementadas, a instrução **blt <rs1>**,

<rs2>, **valor_imediato**. Para a sua implementação altere o que for necessário diretamente no caminho de dados monociclo e na tabela da UC vista em sala de aula. Caso você conclua que não precisam ser alterados, mostre porque definindo as conexões usadas para a execução da instrução.

Para a instrução **blt** temos um formato de instrução similar à **beq**, já implementada no caminho de dados, onde com base no valor imediato, incrementamos o valor **PCNext = PC** (calculado em **PCTarget**).

Para a instrução na UC, temos

- Sem escrita de registradores, portanto **RegWrite = 0**
- Valor imediato tipo **B**, portanto **ImmSrc = 10**
- O comparativo é feito entre registradores, logo **ALUSrc = 0**
- Não há escrita em memória, logo **MemWrite = 0**
- Não há utilização do retorno para escrita em registradores, logo **ResultSrc = X**
- Como resultado, temos um desvio, logo **Branch = 1**
- Para comparar **blt** comparamos os valores em cada registrador **SrcA - SrcB**, logo **ALUOp = 01**

Instruction	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp	Jump
blt	0	10	0	0	XX	1	01	0